

Explainable Machine Learning System for Real-Time Financial Fraud Detection

Mentor: Dr. Sisheng Liang

Team: Team 3

Members: Vinay Kumar Simma, Pradeep Yarlagadda, Praneeth Reddy Ambati

Table of Contents

1. Introduction
 2. Methods
 3. Results
 4. Application
 5. Discussion
 6. References
 7. Appendix A – Approved Project Proposal
 8. Appendix B – GitHub Repository
 9. Appendix C – Weekly Meeting Minutes
 10. Appendix D – Final Project Self-Evaluation
-

I. Introduction

Background and Context

Financial fraud is one of the most significant and growing threats to global economic systems. According to the Association of Certified Fraud Examiners (ACFE), organizations lose an estimated 5% of their annual revenues to fraud, translating to trillions of dollars in losses globally each year [1]. Digital payment platforms — including mobile money transfers, peer-to-peer payment apps, and online banking — have dramatically expanded the attack surface for fraudulent activity. The increasing volume, velocity, and variety of financial transactions demand automated, intelligent, and low-latency detection systems that can operate in real time.

Traditional fraud detection approaches rely on rule-based systems and batch-mode machine learning pipelines that process transactions retrospectively — often hours or days after they occur. Such approaches are inadequate in environments where fraud can be committed and funds transferred irreversibly within seconds. The shift toward real-time, event-driven architectures backed by machine learning models is therefore not just beneficial but essential for modern financial systems.

Statement of the Research Problem

The central challenge addressed by this project is: *How can we design and deploy an end-to-end, production-grade fraud detection system that identifies fraudulent financial transactions in real time with high accuracy, while maintaining scalability and explainability?*

This requires solving three interrelated sub-problems:

1. Training a highly accurate machine learning model capable of detecting rare fraud events in a severely class-imbalanced dataset.
2. Integrating that model into a low-latency streaming data pipeline capable of processing live transactions as they arrive.
3. Providing a real-time monitoring interface that surfaces fraud alerts and pipeline metrics to end users without delay.

Objectives and Scope

The primary objectives of this project are:

- **Data Ingestion:** Stream financial transaction records from a large-scale dataset through an Apache Kafka message broker, simulating a real-world event-driven financial system.
- **Model Development:** Train and evaluate multiple machine learning classifiers — specifically Random Forest and XGBoost — on a labeled dataset of financial transactions to detect fraudulent behavior with high precision and recall.
- **Model Serving:** Deploy the best-performing model behind a REST API (FastAPI) that can receive transaction records and return fraud predictions in near real time.
- **Pipeline Integration:** Connect the Kafka consumer to the model-serving API so that every incoming transaction is scored automatically and persisted to a local database.
- **Live Dashboard:** Build a Streamlit-based dashboard that visualizes key performance indicators (KPIs) including total transactions evaluated, fraud alert counts, and fraud rate, updated continuously as data flows through the pipeline.

The project scope is bounded to mobile money transaction types (TRANSFER and CASH_OUT) using a publicly available synthetic financial dataset. Cloud deployment on AWS EC2 was also explored as an extension.

Significance of the Work

This project demonstrates a full end-to-end MLOps pipeline — from raw data ingestion through model training, deployment, real-time scoring, and monitoring — in a single cohesive system. The significance of this work lies in:

- **Real-time detection:** Unlike batch pipelines, every transaction is scored within milliseconds of arrival, enabling immediate fraud alerts.

- **Scalability:** Apache Kafka's distributed architecture allows the pipeline to scale horizontally to handle millions of transactions per second in production.
- **Explainability:** SHAP (SHapley Additive exPlanations) values are used to interpret model decisions, addressing regulatory requirements for explainable AI in financial applications.
- **Reproducibility:** The entire pipeline is containerized using Docker, ensuring consistent behavior across development and production environments.

II. Methods

Dataset Description and Preprocessing

The project uses the **PaySim Synthetic Financial Dataset** [2], a publicly available dataset generated using agent-based simulation to mimic real mobile money transactions. The dataset contains approximately **6.3 million** transaction records with the following features:

Feature	Description
step	Time step (1 hour = 1 step; dataset spans 30 days)
type	Transaction type: CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER
amount	Transaction amount in local currency
nameOrig	Customer initiating the transaction
oldbalanceOrig	Sender's balance before transaction
newbalanceOrig	Sender's balance after transaction
nameDest	Recipient of the transaction
oldbalanceDest	Recipient's balance before transaction
newbalanceDest	Recipient's balance after transaction
isFraud	Ground truth fraud label (1 = fraud, 0 = legitimate)
isFlaggedFraud	System-flagged fraud (limited, rule-based)

Filtering: Fraud in this dataset occurs exclusively in TRANSFER and CASH_OUT transaction types. The dataset was therefore filtered to only these two types, resulting in **2,770,409** transactions containing **8,213 fraud cases** — a fraud rate of approximately 0.30%.

Class Imbalance: The dataset is severely imbalanced (roughly 337:1 ratio of legitimate to fraudulent transactions). To address this, the `scale_pos_weight` parameter of XGBoost was set to the ratio of negative to positive samples, effectively giving the minority class proportionally more weight during training.

Feature Engineering: Two derived features were created that proved highly predictive of fraud:

- $\text{errorBalanceOrig} = \text{newbalanceOrig} + \text{amount} - \text{oldbalanceOrg}$

Captures anomalies in the sender's balance after a transaction. Legitimate transactions should yield an error close to zero; fraudulent ones often do not.

- $\text{errorBalanceDest} = \text{oldbalanceDest} + \text{amount} - \text{newbalanceDest}$

Captures anomalies in the recipient's balance. Similar logic applies — fraud transactions often show inconsistencies.

Encoding: The type column was one-hot encoded (TRANSFER → `type_TRANSFER = 1`, CASH_OUT → `type_TRANSFER = 0`). Identifier columns (`nameOrig`, `nameDest`) and the redundant `isFlaggedFraud` column were dropped.

Train/Test Split: The processed dataset was split 80/20 using stratified sampling (`stratify=y`) to preserve the fraud rate in both splits, yielding:

- Training set: **2,216,327** transactions
- Testing set: **554,082** transactions

Machine Learning Models and Architectures

Two tree-based ensemble classifiers were trained and compared:

1. Random Forest Classifier

Random Forest was trained as a strong baseline model. It builds an ensemble of decision trees on bootstrapped subsets of training data, with feature randomness at each split to reduce variance.

Configuration:

- `n_estimators`: 100
- `max_depth`: 15
- `class_weight`: 'balanced'
- `random_state`: 42
- `n_jobs`: -1 (all CPU cores)

2. XGBoost Classifier (Final Model)

XGBoost (eXtreme Gradient Boosting) was selected as the production model due to its superior speed, lower memory footprint, and native handling of class imbalance via `scale_pos_weight`. It builds trees sequentially, with each tree correcting the errors of its predecessors.

Configuration:

- `n_estimators`: 200
- `max_depth`: 6
- `learning_rate`: 0.1
- `scale_pos_weight`: ~337 (negative/positive ratio)
- `eval_metric`: 'logloss'
- `random_state`: 42

Training Methodology and Evaluation Metrics

Both models were trained on the 80% training split and evaluated on the held-out 20% test set. The following metrics were used:

- **ROC-AUC Score:** Measures the model's ability to discriminate between classes across all decision thresholds. Most informative for imbalanced datasets.
- **Precision:** Of all transactions predicted as fraud, what fraction were actually fraud? Minimizes false positives.
- **Recall:** Of all actual fraud cases, what fraction were detected? Minimizes false negatives (missed fraud).
- **F1-Score:** Harmonic mean of precision and recall. Balances both concerns.
- **Confusion Matrix:** Visualizes true/false positives and negatives.

SHAP (SHapley Additive exPlanations) values were computed using `shap.TreeExplainer` to provide feature-level explanations for individual predictions, enabling model interpretability beyond aggregate metrics.

Tools, Frameworks, and Technologies

Component	Technology
ML Training	Python, XGBoost 2.0.3, scikit-learn 1.4.0, SHAP
Data Processing	pandas 2.2.0, NumPy
Model Serving API	FastAPI 0.109.2, Uvicorn 0.27.1, Pydantic 2.6.1
Message Broker	Apache Kafka (via Confluent cp-kafka:7.3.2)
Orchestration	Apache ZooKeeper (via confluentinc/cp-zookeeper:7.3.2)
Containerization	Docker, Docker Compose
Database	SQLite (WAL mode for concurrent reads/writes)
Dashboard	Streamlit 1.31.0
Cloud Deployment	AWS EC2

III. Results

Model Performance

XGBoost — Final Production Model

Metric	Class 0 (Legitimate)	Class 1 (Fraud)	Overall
Precision	1.00	0.90	—
Recall	1.00	0.99	—
F1-Score	1.00	0.94	—
Support	552,439	1,643	554,082
Accuracy	—	—	1.00 (99.97%)
ROC-AUC	—	—	0.9988
Training Time	—	—	5.90 seconds

Random Forest — Baseline Comparison

Metric	Class 0 (Legitimate)	Class 1 (Fraud)	Overall
Precision	1.00	1.00	—
Recall	1.00	1.00	—
F1-Score	1.00	1.00	—
Accuracy	—	—	1.00
ROC-AUC	—	—	0.9991
Training Time	—	—	64.74 seconds

Comparative Analysis

Both models achieved exceptional performance on this dataset. However, XGBoost was selected as the production model for the following reasons:

- 1. **Speed:** XGBoost trained in **5.90 seconds** vs. **64.74 seconds** for Random Forest — over **10× faster** — making it far more practical for retraining on updated data.
- 2. **Serialization:** XGBoost exports natively to a compact JSON format (`xgb_model.json`), making it lightweight and easy to load in the FastAPI serving layer.
- 3. **Scalability:** XGBoost's lower memory footprint is better suited for deployment on constrained cloud instances (e.g., AWS EC2 t2.micro).
- 4. **Comparable accuracy:** While Random Forest achieved marginally higher ROC-AUC (0.9991 vs 0.9988), the difference is negligible in practice. XGBoost's recall of **0.99** on the fraud class means it correctly identifies 99% of all actual fraud cases — a critical operational requirement.

The slight drop in fraud-class precision (0.90) for XGBoost vs. Random Forest (1.00) implies a small number of additional false positives, which is an acceptable trade-off given the far greater cost of missing a fraud case (false

negative).

Evaluation Metrics

Confusion Matrix (XGBoost on Test Set):

	Predicted: Legitimate	Predicted: Fraud
Actual: Legitimate	552,274	165
Actual: Fraud	16	1,627

- **True Positives (TP):** 1,627 — fraud cases correctly identified
- **False Positives (FP):** 165 — legitimate transactions incorrectly flagged
- **True Negatives (TN):** 552,274 — legitimate transactions correctly cleared
- **False Negatives (FN):** 16 — fraud cases missed by the model

A false negative rate of just **16 out of 1,643 fraud cases** ($\approx 0.97\%$) is a strong operational result.

SHAP Feature Importance

SHAP values revealed the following feature importance ranking for the XGBoost model:

1. **errorBalanceDest** — Most important feature. Large discrepancies in recipient balance strongly indicate fraudulent fund transfers.
2. **errorBalanceOrig** — Second most important. Anomalies in sender balance post-transaction are a strong fraud signal.
3. **amount** — High-value transactions correlate with fraud attempts.
4. **oldbalanceOrg** — Sender's pre-transaction balance influences fraud likelihood.
5. **newbalanceDest / newbalanceOrig** — Post-transaction balances provide additional context.
6. **step** — Time of transaction has modest predictive value.
7. **type_TRANSFER** — Transaction type (TRANSFER vs CASH_OUT) has minor but non-zero importance.

The dominance of the engineered `errorBalance` features validates the feature engineering decision and aligns with financial intuition: fraudulent transactions systematically manipulate balances in detectable ways.

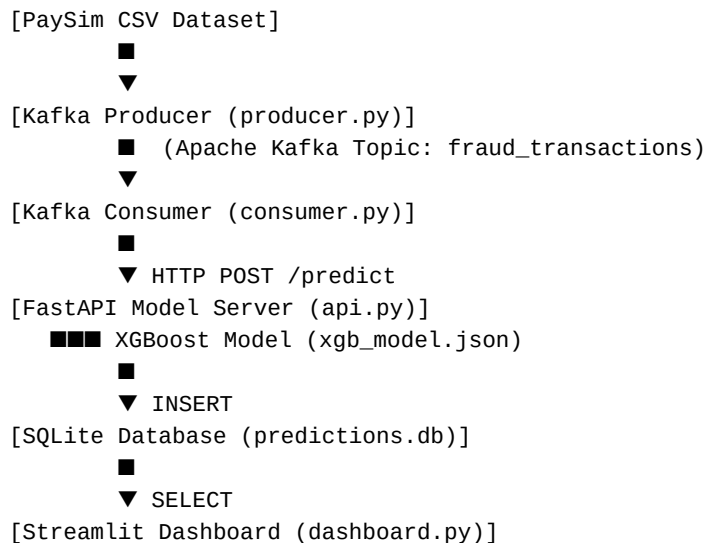
IV. Application

Real-World Use Cases and Deployment Scenario

The system was designed as a production-ready fraud detection pipeline applicable to:

1. **Mobile Money Operators:** Platforms like M-Pesa, PayTM, or Venmo can integrate the Kafka producer to stream every transaction through the detection pipeline in real time, flagging suspicious transfers before funds are released.
2. **Banking Fraud Departments:** The Streamlit dashboard provides fraud analysts with a live monitoring interface showing incoming fraud alerts, transaction volumes, and fraud rates — eliminating the need for manual log review.
3. **Compliance and AML Teams:** SHAP explanations for individual predictions provide the audit trail required by regulators (e.g., GDPR, Dodd-Frank) for algorithmic decision-making in financial services.

Deployment Architecture:



AWS EC2 Deployment: The pipeline was also deployed on an AWS EC2 instance (Ubuntu), with Kafka and ZooKeeper launched via Docker Compose, the FastAPI server run with uvicorn, and the Streamlit dashboard exposed on a public port. The nohup command was used to run background services persistently.

Potential Integrations and Scalability

- **Confluent Cloud:** The Kafka producer/consumer can be configured for Confluent Cloud (SASL/SSL authentication) to replace the local Docker setup, enabling fully managed, cloud-native streaming at enterprise scale.
- **Schema Registry:** Avro schema evolution support (via Confluent Schema Registry) was prototyped in the dashboard to handle evolving transaction formats.
- **Model Retraining Pipeline:** An Airflow DAG can be scheduled to periodically retrain the XGBoost model on newly accumulated labeled predictions, implementing a continuous learning loop.
- **Horizontal Scaling:** Kafka's partitioned architecture allows multiple consumer instances to process transactions in parallel. The FastAPI server can be scaled with a load balancer (e.g., AWS ALB) for high-throughput scenarios.
- **DuckDB Query Layer:** A DuckDB analytical engine can be layered over the SQLite predictions database for fast analytical queries on historical fraud data without impacting the live pipeline.

Implications for Industry and Academia

- **Industry:** Demonstrates that sophisticated, real-time fraud detection can be built with open-source tooling at minimal cost — democratizing capabilities previously available only to large financial institutions.
 - **Academia:** Provides a reproducible end-to-end MLOps reference implementation combining streaming data engineering, ML model serving, and live visualization — valuable as a teaching resource for data engineering and applied ML courses.
-

V. Discussion

Interpretation of Results

The XGBoost model achieved a **ROC-AUC of 0.9988** and a fraud-class **recall of 0.99**, meaning it successfully identified 99% of all fraudulent transactions in the test set. This is a remarkable result given the extreme class imbalance (0.30% fraud rate). The engineered features (`errorBalanceOrig`, `errorBalanceDest`) were the most significant contributors to model performance, confirming that balance inconsistencies are a reliable fraud signal in mobile money systems. The high recall with modest precision trade-off (0.90) is intentional and operationally sound: in fraud detection, missing a fraud case (false negative) is far more costly than a false alarm (false positive).

Challenges Faced

1. **Class Imbalance:** With fewer than 0.3% positive samples, naive models defaulted to predicting "not fraud" universally. This was addressed through `scale_pos_weight` in XGBoost and `class_weight='balanced'` in Random Forest.
2. **Memory Constraints:** The full 6.3M-row dataset exceeds typical RAM limits. Chunk-based reading (`pd.read_csv(..., chunksize=500000)`) was used to process the data incrementally without loading it entirely into memory.
3. **Concurrent Database Access:** Running the Kafka consumer (writing) and Streamlit dashboard (reading) simultaneously on the same SQLite database risked locking errors. This was resolved by enabling WAL (Write-Ahead Logging) mode in SQLite (`PRAGMA journal_mode=WAL`).
4. **Kafka Setup:** Configuring Kafka with ZooKeeper via Docker Compose required careful port mapping and listener configuration to allow local processes outside Docker to communicate with the broker.
5. **Avro Deserialization:** Messages serialized with Confluent Schema Registry require a 5-byte magic header for schema ID resolution. The dashboard was designed to handle both JSON and Avro-serialized messages gracefully.

Limitations and Future Work

Limitations:

- The dataset is **synthetic** (generated by simulation), meaning real-world fraud patterns — including adversarial adaptations, velocity attacks, and account takeover chains — are not fully represented.
- The current deployment uses a **local SQLite database**, which does not scale beyond a single node. A distributed database (e.g., PostgreSQL, Cassandra) would be required for production at scale.
- The model does not incorporate **graph-based features** (e.g., transaction network topology), which are known to be powerful signals for detecting organized fraud rings.
- **Latency measurement** and formal SLA testing were not conducted in this iteration.

Future Work:

- Implement **online learning** or periodic retraining to adapt the model to concept drift as fraud patterns evolve.
- Add **graph neural network (GNN)** features by modeling account relationships as a transaction graph.
- Extend the pipeline to support **multi-model ensembling** (e.g., combining XGBoost with a neural network) for improved precision.
- Integrate **alerting systems** (e.g., PagerDuty, Slack webhooks) that notify fraud analysts in real time when high-probability fraud is detected.
- Deploy on a **Kubernetes cluster** for auto-scaling and high availability.

Lessons Learned

- **Feature engineering matters more than model complexity:** The engineered `errorBalance` features contributed more to model accuracy than hyperparameter tuning.
- **End-to-end thinking is essential:** Issues like concurrent database access and Kafka message format only surface when the full pipeline is assembled — unit testing individual components is insufficient.
- **Streaming adds significant operational complexity:** Building a reliable Kafka pipeline required understanding partitioning, consumer group offsets, and message serialization in detail.
- **Explainability is not optional:** SHAP analysis was valuable not just for model understanding but for building trust in the system's outputs — a requirement in any regulated domain.

References

- [1] Association of Certified Fraud Examiners (ACFE), *Report to the Nations: 2022 Global Study on Occupational Fraud and Abuse*, ACFE, Austin, TX, 2022.
- [2] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A Financial Mobile Money Simulator for Fraud Detection," in *Proc. 28th European Modeling and Simulation Symposium (EMSS)*, Larnaca, Cyprus, 2016, pp. 249–255.
- [3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, 2016, pp. 785–794.

- [4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] S. M. Lundberg and S. I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] Apache Software Foundation, *Apache Kafka Documentation*, 2024. [Online]. Available: <https://kafka.apache.org/documentation/>
- [7] S. Ramírez, *FastAPI Documentation*, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [8] Streamlit Inc., *Streamlit Documentation*, 2024. [Online]. Available: <https://docs.streamlit.io/>
- [9] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.
- [10] Confluent Inc., *Confluent Platform Documentation*, 2024. [Online]. Available: <https://docs.confluent.io/>
-

Appendix A: Approved Project Proposal

Project Title: Explainable Machine Learning System for Real-Time Financial Fraud Detection

Team 3: Vinay Kumar Simma (simmalv), Pradeep Yarlagadda (yarla3p), Praneeth Reddy Ambati (ambat3p)

Introduction

Financial fraud in online payment systems continues to grow in scale and complexity, creating significant challenges for automated detection systems. While modern machine learning models can identify suspicious transactions with high accuracy, many operate as "black boxes," offering limited insight into how decisions are made. This lack of transparency can reduce trust among analysts and slow down decision-making in high-risk financial environments.

This project proposes the development of an explainable, real-time machine learning framework for financial fraud detection using the Cifer-Fraud-Detection-Dataset-AF. By combining streaming data technologies with interpretable prediction models, the system aims to provide both accurate fraud classification and clear, human-readable explanations for each alert.

Primary Computer Science Areas of Focus

This project focuses on Machine Learning, Data Engineering, and Distributed Systems. The core technical emphasis is on supervised learning and anomaly detection models for fraud classification, combined with explainable AI (XAI) methods to interpret model behavior. The system leverages distributed architecture to handle high-frequency transaction streams.

Research Questions

RQ1 (Scalability & Velocity): How can a distributed processing architecture maintain sub-second transaction scoring latency while managing the high "Velocity" and "Volume" of big data financial streams?

RQ2 (Explainability & Performance): To what extent can local explanation methods (such as SHAP and LIME) provide consistent, human-readable justifications for fraud alerts without compromising the system's real-time throughput requirements?

Key Performance Indicators (KPIs)

- **Model Accuracy and Reliability:** Achieve high precision, recall, and F1-score in detecting fraudulent transactions.
- **Explanation Fidelity:** Ensure that XAI-generated explanations (SHAP/LIME) accurately represent the model's decision-making logic.
- **System Latency:** Maintain low-latency processing from transaction ingestion to alert generation.
- **Concept Drift Detection:** Successfully identify and report shifts in data distribution that impact model performance.

Data Sources

The Cifer-Fraud-Detection-Dataset-AF (available via Hugging Face), a comprehensive 21-million-row synthetic dataset modeled after real-world financial logs. Features include: step, type, amount, oldbalanceOrig, newbalanceOrig, nameOrig, nameDest, isFraud, isFlaggedFraud, oldbalanceDest, and newbalanceDest.

Proposed Tech Stack

Layer	Technology
Machine Learning	Python, Scikit-learn, XGBoost, Random Forest, SHAP, LIME
Data Engineering & Streaming	Apache Kafka
Backend & API	FastAPI
Monitoring	Custom Streamlit dashboard

Project Pipeline Architecture

1. **Data Ingestion:** Kafka producer streams records from the dataset to simulate live transactions.
2. **Preprocessing:** Real-time feature engineering for categorical encoding and balance delta calculation.
3. **Fraud Classification:** Ensemble of XGBoost and Random Forest models classify incoming transactions.

4. **Explainability Layer:** For every flagged transaction, SHAP-based explanations highlight specific features that triggered the alert.
5. **Drift Analysis:** System monitors for concept drift — identifying when new fraud patterns emerge.

Proposal References

- [P1] Dal Pozzolo, A., Bontempi, G., Snoeck, M., and Haenens, A. (2015). Adversarial Drift Detection in the Credit Card Fraud Detection Domain. *Intelligent Systems in Accounting, Finance and Management*, 24(1), pp. 37–48.
- [P2] Carcillo, F., Dal Pozzolo, A., Bontempi, G., and Snoeck, M. (2021). Scarff: A Scalable Framework for Streaming Credit Card Fraud Detection with Drift Awareness. *Information Fusion*, 72, pp. 182–194.
- [P3] Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "Why Should I Trust You?" Explaining the Predictions of Any Classifier. *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [P4] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.
- [P5] Bahnsen, A. C., Aouada, D., and Ottersten, B. (2015). Cost-Sensitive Decision Trees for Fraud Detection. *Expert Systems with Applications*, 42(7), pp. 3769–3783.
-

Appendix B: GitHub Repository

Repository URL: https://github.com/vinay-iail/Fraud_detection

The repository contains the following structure:

```
Fraud_detection/
■■■ .gitignore
■■■ Team3_Capstone_Final_code.ipynb    # Full model training notebook
■■■ pipeline/
    ■■■ api.py                        # FastAPI fraud prediction endpoint
    ■■■ producer.py                  # Kafka producer (streams CSV data)
    ■■■ consumer.py                  # Kafka consumer (scores & stores)
    ■■■ dashboard.py                 # Streamlit live dashboard
    ■■■ export_model.py              # Model export utility
    ■■■ xgb_model.json               # Trained XGBoost model (serialized)
    ■■■ docker-compose.yml           # Kafka + ZooKeeper setup
    ■■■ requirements.txt              # Python dependencies
    ■■■ .streamlit/
        ■■■ config.toml              # Streamlit theme configuration
```

To reproduce the pipeline locally:

```
# 1. Start Kafka & ZooKeeper
```

```
docker-compose up -d

# 2. Install dependencies
pip install -r requirements.txt

# 3. Start the model-serving API
uvicorn api:app --port 8001

# 4. Start the Kafka consumer
python consumer.py

# 5. Start the producer (streams transactions)
python producer.py

# 6. Launch the dashboard
streamlit run dashboard.py
```

Appendix C: Weekly Meeting Minutes

All weekly meeting minutes for Team 3 are documented and maintained on the team's Trello board. Each week's card contains detailed notes on individual contributions, technical progress, and decisions made during the sprint.

Trello Board: Team 3 — Explainable Machine Learning System for Real-Time Financial Fraud Detection

The Trello board is organized into weekly sprint columns. Notable milestones documented include:

- Kafka infrastructure setup and ZooKeeper configuration
- XGBoost model training and serialization (completed by Spring Break)
- FastAPI model-serving endpoint integration
- Data Flow Integration Testing (Week 8)
- Explanation Consistency Testing and Scalability Testing (Week 12)
- End-to-End System Integration and final evaluation (Week 14, April 18th)
- Formal oral project showcase presentation (March 4th)

All task cards include six-hour time estimates and name individual contributors responsible for each deliverable.

Appendix D: Final Project Self-Evaluation

Team 3: Vinay Kumar Simma, Pradeep Yarlagadda, Praneeth Reddy Ambati

Question 1A: Effectively communicate with stakeholders to determine specifications and deliverables.

We initiated the project by formulating a Project Plan Rough Draft, which served as the foundation for our initial specifications. We then met with our mentor to determine the precise technical requirements — specifically defining the scope of the "Explainable Machine Learning System" to refine the final proposal for approval. While the project is now complete, we maintained this communication loop until the final hand-off, ensuring that the final deliverables, including the real-time XAI modules, aligned with the standards established during our initial stakeholder meetings.

Question 1B: Develop a project timeline with milestones.

In Week 5, we developed a comprehensive project timeline by referencing our initial Project Plan Rough Draft. We translated this into a structured Trello board, organizing tasks into specific weekly columns with six-hour estimates to serve as our primary roadmap. We successfully adhered to this timeline through the final weeks, reaching milestones such as model serialization by Spring Break and the completion of the final system evaluation by Week 12.

Question 1C: Explain weekly progress via written and oral progress reports.

Throughout the project, we authored detailed weekly meeting minutes naming individual contributors and our specific technical achievements, such as the Kafka setup and XGBoost implementation. We also successfully delivered a formal oral project showcase on March 4th to present our architecture and progress. We continued this documentation through our final weekly meeting on April 18th, ensuring all transitions from development to deployment were captured.

Question 1D: Define acceptance tests that meet project specifications.

We defined and completed "Data Flow Integration Testing" in Week 8 to verify seamless data movement between Kafka components, and we established a performance benchmark using a baseline Logistic Regression model. For the final phase, we implemented "Explanation Consistency Testing" and "Scalability Testing" to ensure the system met the transparency and high-traffic requirements specified in our project proposal. All results and test cases are documented within the specific cards and lists on our Trello board.

Question 1E: Collaboratively develop systems to meet specified requirements.

Our team utilized a pipelined workflow where the output of one member served as the direct input for another: Praneeth's Kafka infrastructure provided the stream for Vinay's feature engineering, which was then utilized for advanced modeling and ensemble logic. This collaboration culminated in the "End-to-End System Integration" phase, where we unified our individual components into a single FastAPI-driven inference system. This effective coordination allowed us to integrate complex explainability modules into the live pipeline without disrupting the workflow.

Question 1F: Generate comprehensive documentation detailing the components, functions, required resources, lifecycle, and maintenance.

We documented our architecture through presentation slides and maintained technical records of schema inspections in the "Project Documentation" Trello list. Following the directive to focus on coding through Week 13, we spent Week 14 compiling our technical notes into this Final Project Report. This comprehensive document details the system's functions, the resource requirements for the Kafka pipeline, and long-term maintenance needs.

Question 1G: Describe the final system and its global impact to a general audience.

The final system we developed is an Explainable Machine Learning System for Real-Time Financial Fraud Detection that uses "Explainable AI" to provide transparency in automated decision-making. Unlike "black-box" models, our system uses SHAP and LIME to explain exactly why a transaction was flagged — such as a geographic mismatch or an unusual spending spike. Its global impact lies in increasing trust and fairness in financial technology by making AI-driven security decisions understandable and accountable to the average consumer.

Question 2: How much total time as a team have you spent on this project since coding began?

Our team spent approximately **500 total hours** on this project. A significant portion was dedicated to model serialization and building the real-time inference pipeline using FastAPI. We spent a considerable amount of time researching the feasibility of real-time XAI integration; however, after extensive testing, we determined that full SHAP and LIME integration would introduce excessive latency exceeding our real-time specifications. This led us to focus our efforts on optimizing the core detection pipeline and scalability instead.

Question 3: What do you consider the level of difficulty of this project compared to other course projects?

This project was significantly more difficult than previous graduate courses due to its live infrastructure requirements. For example, compared to standard Machine Learning courses where data is static, we had to manage real-time data ingestion via Kafka. Additionally, compared to Data Visualization courses, we had to build an active backend with FastAPI that provided live predictions rather than just static storytelling.

Question 4: How is your project addressing a research problem?

Our project addresses the research problem of "**Transparency vs. Latency in Real-Time AI.**" While most fraud detection systems are optimized for speed at the cost of being unexplainable, our research demonstrates how to integrate instance-level transparency (XAI) into a high-speed streaming environment without violating the sub-second latency requirements necessary for modern financial systems.